

```

1  (*
2                                     CS 51 Lab 19
3                                     Conway's Game of Life Cellular Automaton
4  *)
5  (*
6                                     SOLUTION
7  *)
8  module G = Graphics ;;
9
10 (* Automaton parameters *)
11 let cGRID_SIZE = 100 ;;           (* width and height of grid in cells *)
12 let cSPARSITY = 5 ;;              (* inverse of proportion of cells initially live *)
13 let cRANDOMNESS = 0.00001 ;;      (* probability of randomly modifying a cell *)
14
15 (* Rendering parameters *)
16 let cCOLOR_LIVE = G.rgb 93 46 70 ;; (* color to depict live cells *)
17 let cCOLOR_DEAD = G.rgb 242 227 211 ;; (* background color *)
18 let cCOLOR_LEGEND = G.rgb 173 106 108 ;; (* color for textual legend *)
19 let cSIDE = 8 ;;                  (* width and height of cells (in pixels) *)
20 let cRENDER_FREQUENCY = 1         (* how frequently grid is rendered (in ticks) *) ;;
21
22 (* Font specification for rendering the legend. OCaml font handling is
23    platform dependent, so we leave this as 'None', but we provide the
24    functionality for use in the solution set. Feel free to play with
25    this or leave it as is. *)
26 let cFONT = Some "-adobe-times-bold-r-normal--34-240-100-100-p-177-iso8859-9"
27
28
29 (*-----
30    The Game of Life
31    *)
32
33 (*.....
34    Game of Life states and updates
35    *)
36
37 (* We take the opportunity to abstract the cell states as an
38    enumerated type *)
39 type life_state =
40   | Live
41   | Dead ;;
42
43 (* flipped state -- Returns the opposite state to 'state'. *)
44 let flipped (state : life_state) : life_state =

```

```

45     match state with
46     | Live -> Dead
47     | Dead -> Live ;;
48
49     (* offset index off -- Returns the `index` offset by `off` allowing
50        for wraparound. *)
51     let rec offset (index : int) (off : int) : int =
52         if index + off < 0 then
53             cGRID_SIZE - (offset ~-index ~-off)
54         else
55             (index + off) mod cGRID_SIZE ;;
56
57     (* life_update grid i j -- Returns the updated value for cell at `i`,
58        `j` in the grid based on rules of Conway's Game of Life. In this
59        implementation, after updating as per the standard GoL update rule,
60        a cell's state is then flipped with probability given by
61        cRANDOMNESS. *)
62     let life_update (grid : life_state array array) (i : int) (j : int)
63         : life_state =
64
65         (* We give a few variant update rules for the Game of Life. These
66            lists are indexed by adjacency count to give the generated cell
67            status.
68
69                0      1      2      3      4      5      6      7      8
70                |      |      |      |      |      |      |      |      |
71                v      v      v      v      v      v      v      v      v *)
72         (* B3/S23: Conway's original game of life *)
73         let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Dead; Dead; Dead] in
74         let live_update = [Dead; Dead; Live; Live; Dead; Dead; Dead; Dead; Dead] in
75
76         (* B3/S12345: https://conwaylife.com/wiki/OCA:Maze
77            let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Dead; Dead; Dead] in
78            let live_update = [Dead; Live; Live; Live; Live; Live; Dead; Dead; Dead] in
79            *)
80         (* B36/S125: https://conwaylife.com/wiki/OCA:2x2
81            let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Live; Dead; Dead] in
82            let live_update = [Dead; Live; Live; Dead; Dead; Live; Dead; Dead; Dead] in
83            *)
84         (* B3678/34678: https://conwaylife.com/wiki/OCA:Day\_%26\_Night
85            let dead_update = [Dead; Dead; Dead; Live; Dead; Dead; Live; Live; Live] in
86            let live_update = [Dead; Dead; Dead; Live; Live; Dead; Live; Live; Live] in
87            *)
88
89     let neighbors = ref 0 in
90     for i' = ~-1 to ~+1 do
91         for j' = ~-1 to ~+1 do

```

```

91     let i_ind = offset i i' in
92     let j_ind = offset j j' in
93     neighbors := !neighbors
94               + match grid.(i_ind).(j_ind) with
95                 | Live -> 1
96                 | Dead -> 0
97     done
98 done;
99 (* determine new state based on neighbor count *)
100 let new_state =
101     match grid.(i).(j) with
102     | Live -> List.nth live_update (!neighbors - 1)
103     | Dead -> List.nth dead_update !neighbors in
104 (* randomly flip an occasional cell state *)
105 if Random.float 1.0 <= cRANDOMNESS then
106     flipped new_state
107 else
108     new_state ;;
109
110 (* life_random_state () -- Returns a random cell state. *)
111 let life_random_state () =
112     let states = [Live; Dead] in
113     List.nth states (Random.int (List.length states))
114
115 (*.....*)
116 Making the Game of Life
117 *)
118
119 (* GoL automaton specification *)
120
121 module LifeSpec : (Cellular.AUT_SPEC
122                   with type state = life_state) =
123 struct
124     type state = life_state
125     let initial : state = Dead
126     let grid_size : int = cGRID_SIZE
127     let random_state = life_random_state
128     let update = life_update
129     let name : string = "Conway's Life"
130     let cell_color (cell : state) : G.color =
131         match cell with
132         | Live -> cCOLOR_LIVE
133         | Dead -> cCOLOR_DEAD
134     let side_size : int = cSIDE
135     let legend_color : G.color = cCOLOR_LEGEND
136     let font : string option = cFONT

```

```

137     let render_frequency : int = cRENDER_FREQUENCY
138 end ;;
139
140 (* GoL automaton *)
141
142 module Aut = Cellular.Automaton (LifeSpec) ;;
143
144 (*.....
145 Running the automaton and displaying the results
146 *)
147
148 (* main seed -- Runs the automaton using the provided random 'seed'
149 (for replicability), displaying updates to the graphics window. *)
150 let main seed =
151
152     Random.init seed;      (* initialize randomness *)
153     Aut.graphics_init ();  (* ... and graphics window *)
154
155     let initial_grid = Aut.random_grid () in
156     Aut.run_grid initial_grid ;;
157
158 (* Run the automaton with user-supplied seed *)
159 let _ =
160     if Array.length Sys.argv <= 1 then
161         Printf.printf "Usage: %s <seed>\n where <seed> is integer seed\n"
162             Sys.argv.(0)
163     else
164         main (int_of_string Sys.argv.(1)) ;;
165

```